

PICCOLO CAFE

# Run it. Understand it. Explain it.

A practical project handbook  
Windows setup + complete workflow tour



Built from the actual React, Fastify and Supabase upgrade. Includes setup commands, four diagrams, code paths, test evidence, troubleshooting and an interview study sequence.

*Original production data and live deployment remain unchanged.*

## Contents

|                                                 |    |
|-------------------------------------------------|----|
| How to use this guide                           | 3  |
| 1. Why GitHub still showed the old version      | 3  |
| 2. What each part does                          | 4  |
| 3. Before your first run                        | 5  |
| 4. Configure .env correctly                     | 6  |
| 5. Provision the database login and start       | 7  |
| 6. Create accounts and your first administrator | 8  |
| 7. Try the visible workflows                    | 8  |
| 8. Read the code in this order                  | 9  |
| 9. Follow one booking from click to commit      | 10 |
| 10. Why simultaneous requests cannot overbook   | 11 |
| 11. Retry keys and private guest access         | 11 |
| 12. States, cancellation and stale edits        | 12 |
| 13. Authentication, authorization and RLS       | 13 |
| 14. Understand the database model               | 14 |
| 15. Menu, images, dashboard and jobs            | 15 |
| 16. Tests and what was actually verified        | 15 |
| 17. Troubleshoot by symptom                     | 16 |
| 18. Deployment, migration and rollback          | 17 |
| 19. Practise a five-minute interview demo       | 18 |
| 20. Check your understanding and keep learning  | 18 |
| Reference pages and commands                    | 19 |

## How to use this guide

You can understand this project by learning it in layers. First run a guest booking. Then follow that request through React, Fastify and PostgreSQL. Finally study why simultaneous requests, retries and permission checks behave correctly. Running the app alone does not establish complete understanding.

- First pass: chapters 1-7. Get the full local application running and try the screens.
- Second pass: chapters 8-15. Trace the code, tables, booking rules and background worker.
- Third pass: chapters 16-20. Reproduce the evidence, troubleshoot, practise the demo and answer the review questions.

Useful starting knowledge: JavaScript functions/promises, React state/effects, HTTP requests, SQL SELECT/INSERT, and basic Git. TypeScript adds checked data contracts; unfamiliar terms are defined in chapter 20. You do not need to learn every library's internal implementation to explain the project's own behavior.

The main application has three operating pieces: a React frontend, a Fastify API, and Supabase. The email worker is optional until you configure real email delivery.

## 1. Why GitHub still showed the old version

The link ending in 34702c85e33e5d6107f24f389edf598fc86da211 opens an immutable old commit. That page will always show that revision, even after new commits are published. A branch page, such as `main` or `upgrade/supabase-reliable-reservations`, shows that branch's current revision.

A file edit is local. A commit saves a local revision. A push publishes that revision to GitHub. A merge brings the upgrade branch into `main`. Deployment is a separate step that depends on the host's settings. These actions are not interchangeable.

In your existing folder, verify where you are:

```
Set-Location 'C:\Users\harib\Desktop\Piccolo Cafe Websitr'  
git branch --show-current  
git status --short  
git log -3 --oneline
```

The active upgrade branch is `upgrade/supabase-reliable-reservations`. The new UI is under `src/app`, and the backend is under `server`. The earlier Firebase code and your earlier UI edits are preserved under `migration/legacy`. Open those only when comparing or migrating the old implementation.

There is also a separate nested checkout named `Piccolo-Cafe-Website`. Work in the root folder shown above. Its independent dirty checkout and the old Firebase hosting cache are not part of the upgrade commit.

For a future clean clone after the branch has been published:

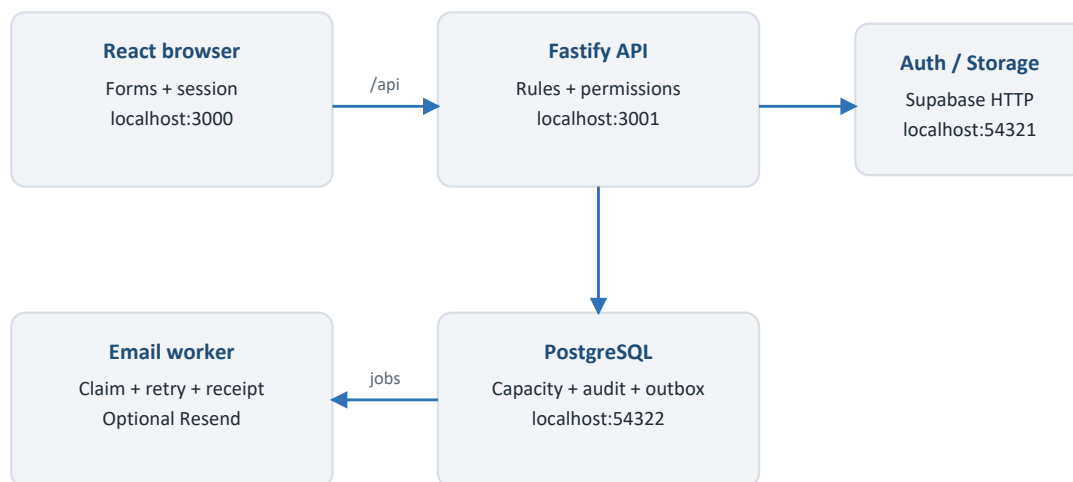
```
git clone --branch upgrade/supabase-reliable-reservations `   
  https://github.com/Ha-r-i/Piccolo-Cafe-Website.git  
Set-Location Piccolo-Cafe-Website
```

The backtick is PowerShell's line continuation character. Keep it as the last character on that line, or put the entire command on one line.

## 2. What each part does

The original application was a React SPA with Firebase Auth/Firestore, browser EmailJS calls and unsigned Cloudinary uploads. The inspected active revision had no MongoDB or backend. The upgrade retains React and replaces those runtime integrations.

| Part                 | Responsibility                                                 | Main location            |
|----------------------|----------------------------------------------------------------|--------------------------|
| React + Vite         | Screens, form state, navigation, API calls                     | src/app                  |
| Fastify + TypeScript | Validate requests, verify identity, authorize operations       | server                   |
| Supabase Auth        | Signup, login, email confirmation, session refresh/reset       | Hosted or local Supabase |
| PostgreSQL           | Persistent data, transactions, capacity, versions, permissions | supabase/migrations      |
| Supabase Storage     | Public menu images uploaded through the staff API              | menu-images bucket       |
| Notification worker  | Deliver durable booking events with retries                    | server/worker.ts         |



Browser also contacts Supabase Auth directly for account sessions.

In development, Vite runs on port 3000. Requests to `/api` are proxied to the Fastify service on port 3001. Supabase's gateway is port 54321, PostgreSQL is 54322 and Studio is 54323. The worker connects to PostgreSQL; it is not an HTTP website.

This is a modular monolith: one backend codebase with auth, menu, reservation, administration and notification modules. A separate worker process exists because sending email can be slow or fail independently. There is no Redis or message broker to operate; durable jobs are stored in PostgreSQL.

### 3. Before your first run

Use the full Supabase setup for login, staff tools and image uploads. The credential-free tests exercise guest workflows with real PostgreSQL, but they do not provide working Supabase Auth, Storage HTTP or the Data API.

Install Node.js 22.12 or newer; Node 24 was used for verification. Install Git and Docker Desktop. Start Docker Desktop and wait until its engine is running. The Supabase CLI uses Docker to run local services. The first startup downloads several images and can take time.

Check tools in a PowerShell terminal in the root repository:

```
node --version
npm --version
git --version
docker info
npm ci
npx supabase start
npx supabase status
```

Expected result: dependency installation finishes, Docker responds, and Supabase reports local URLs and keys. On first use, npx may ask to download the Supabase CLI package. Node 22.12+ is required by the chosen tooling, not just by the application syntax.

The CLI applies the versioned migrations and development seed on a fresh local stack. Do not then replay the same migrations with the independent `db:migrate` runner. Use one migration owner per database.

Keep ports 3000/3001 free. Use `http://localhost:3000` for the frontend so it matches the Auth redirect configuration. This setup uses local development data. Creating a local account or reservation does not update the original Firebase application.

## 4. Configure .env correctly

The existing .env may contain old Cloudinary values. Preserve it before replacing it. This procedure makes a timestamped backup; the backup and the new .env are ignored by Git.

```
if (Test-Path .env) {
  $cafeBackup = ".env.backup-$(Get-Date -Format yyyyMMdd-HH:mm:ss)"
  Copy-Item -LiteralPath .env -Destination $cafeBackup
}
Copy-Item -LiteralPath .env.example -Destination .env
notepad .env
```

Use the URL/public key/secret key printed by `npx supabase status`. Older CLI labels may say `anon` and `service_role`. The public/anon key is intended for the browser. The secret/service\_role key belongs only on the server.

| Variable                      | Local value or source                                     |
|-------------------------------|-----------------------------------------------------------|
| VITE_SUPABASE_URL             | http://127.0.0.1:54321                                    |
| VITE_SUPABASE_PUBLISHABLE_KEY | Local publishable/anon key                                |
| VITE_API_URL                  | /api                                                      |
| SUPABASE_URL                  | http://127.0.0.1:54321                                    |
| SUPABASE_PUBLISHABLE_KEY      | Same local public key                                     |
| SUPABASE_SECRET_KEY           | Local secret/service_role key                             |
| ADMIN_DATABASE_URL            | Local operator DB URL from status; normally postgres user |
| DATABASE_URL                  | Restricted cafe_api URL shown below                       |
| DATABASE_SSL                  | false for this local connection                           |
| WEB_ORIGIN                    | http://localhost:3000                                     |
| GUEST_TOKEN_SECRET            | Stable random secret, at least 32 characters              |
| RATE_LIMIT_SECRET             | A different random secret, at least 32 characters         |

Generate a 64-character hexadecimal secret by running this command three times and keeping the results separate:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Use one result as the runtime DB password, one as GUEST\_TOKEN\_SECRET, and one as RATE\_LIMIT\_SECRET. Hexadecimal passwords avoid special-character URL-encoding problems. Put the runtime password into:

```
DATABASE_URL=postgres://cafe_api:YOUR_PASSWORD@127.0.0.1:54322/postgres
```

Leave `RESEND_API_KEY` and `EMAIL_FROM` empty until you configure an email provider. Keep the remaining local defaults from `.env.example`. Never put a secret in a variable beginning with `VITE_`; those values are bundled into browser JavaScript. Restart development servers after changing environment values.

## 5. Provision the database login and start

The SQL migrations create the `cafe_api` role without login access. Provision its password using the operator connection. Replace the placeholder below with the same random password used in `DATABASE_URL`:

```
$env:CAFE_API_PASSWORD = 'YOUR_RUNTIME_DATABASE_PASSWORD'
npm run db:role
npm run db:role -- --apply
Remove-Item Env:CAFE_API_PASSWORD
npm run dev
```

The first role command validates configuration without changing the password. The `--apply` command writes it. Removing the temporary environment variable does not remove the database password. The API must connect as `cafe_api`; it rejects an owner/superuser URL. `ADMIN_DATABASE_URL` is for operator commands only and must not be configured on deployed API/worker hosts.

Keep this terminal running. `npm run dev` starts frontend and API together. Open a second PowerShell terminal to check readiness:

```
Invoke-RestMethod http://127.0.0.1:3001/api/health/ready
Invoke-RestMethod http://127.0.0.1:3001/api/settings
```

Expected readiness result: status is ready. Then open `http://localhost:3000`. The menu should show clearly labelled development samples. Try a guest booking for tomorrow or a later date at an offered time.

For separate terminals, use `npm start` for the frontend and `npm run dev:api` for the API. `npm start` by itself does not start the backend. Stop development processes with `Ctrl+C`. Stop the local stack with `npmx supabase stop` when finished. This is not a command to delete production data.

## 6. Create accounts and your first administrator

At `/account`, choose signup and use a password of at least 12 characters. Local Supabase requires email confirmation. Open the local mail UI URL printed by `npx supabase status`, find the confirmation email and follow its link. This tests local Auth emails; it does not configure Resend booking emails.

After confirming, open local Supabase Studio at `http://127.0.0.1:54323`. In Authentication/Users, copy the confirmed account's UUID. In another terminal in the repository:

```
npm run admin:bootstrap -- --user-id YOUR_CONFIRMED_USER_UUID
npm run admin:bootstrap -- --user-id YOUR_CONFIRMED_USER_UUID --apply
```

The first command is a dry run. Bootstrap rejects an unconfirmed user and refuses to bootstrap a second administrator. Refresh or sign out/in to refresh the displayed role, then visit `/admin/dashboard`.

For a staff account, sign up and confirm a second user. Use the administrator's Team access screen to assign staff using that UUID. An ordinary customer cannot promote themselves. Database-owned profile roles determine authority; editable Auth metadata is ignored. The last administrator cannot be demoted through the role API.

The account page supports login, signup confirmation, logout, password-reset requests, changing a signed-in user's password and listing that user's reservations. A reset requires the configured mail link and redirect to return to the account page. Complete the reset with the Change password action after the recovery session is established.

Auth operations depend on actual Supabase services. Their implementation exists, but the workspace verification did not establish a successful real Auth round trip; run the real-stack checks before relying on this for a demonstration.

## 7. Try the visible workflows

Start on the home page, browse the menu and use category, dietary and search filters. Clearing filters restores results. Seed content is sample data, including prices; replace it with reviewed cafe content before public use.

Book as a guest at `/booking`. Choose a future date, party size and returned available time. Fill your name, phone and email. A successful request is saved as pending, not automatically confirmed. Save the private booking link. Open it to see the current reservation and cancel before the configured deadline.

The link's token is a credential for that booking. Losing a guest link does not grant recovery by matching an email; that would allow someone else to claim a record. Account bookings instead belong to the verified user who created them. The private link remains usable when its guest signs in later.

In the staff dashboard, search reservations by name/email/phone, filter date/status and page through results. Confirm or reject a pending request. Arrival, no-show and completion actions are time/state restricted. Open audit history to see who changed a booking and its version.

In Menu management, create a category/item, set integer-paise prices, edit and publish/unpublish it. Allowed image uploads go through the staff API and Supabase Storage. In Notifications, view

queued/processing/sent/dead events. Dashboard counts are queried from the database; no revenue or customer-popularity claims are invented.

For a quick complete local acceptance check, confirm: readiness works, menu filters work, a guest request appears in Studio, the private link loads, cancellation releases seats, a confirmed administrator can manage staff tools, a customer cannot, and image uploads show the stored WebP. Provider delivery is an additional check when configured.

## 8. Read the code in this order

| File or area                              | What to look for                                           |
|-------------------------------------------|------------------------------------------------------------|
| package.json                              | Commands and runtime/dev dependencies                      |
| src/app/main.tsx                          | React root, router and Auth provider                       |
| src/app/App.tsx                           | Routes and navigation                                      |
| src/app/api.ts                            | API wrapper, session token and safe client errors          |
| src/app/Booking.tsx                       | Form, offered slots, retry key and private management page |
| server/index.ts                           | Config, restricted DB role check, API startup              |
| server/app.ts                             | Hooks, limits, CORS, request IDs and module registration   |
| server/modules/reservations.ts            | Validation, availability and SQL function calls            |
| supabase/migrations/202610080001_core.sql | Tables, booking invariants, roles and RLS                  |
| server/modules/notifications.ts           | Job claiming, fenced updates and provider call             |
| tests/integration/database.test.ts        | Executable examples of difficult cases                     |

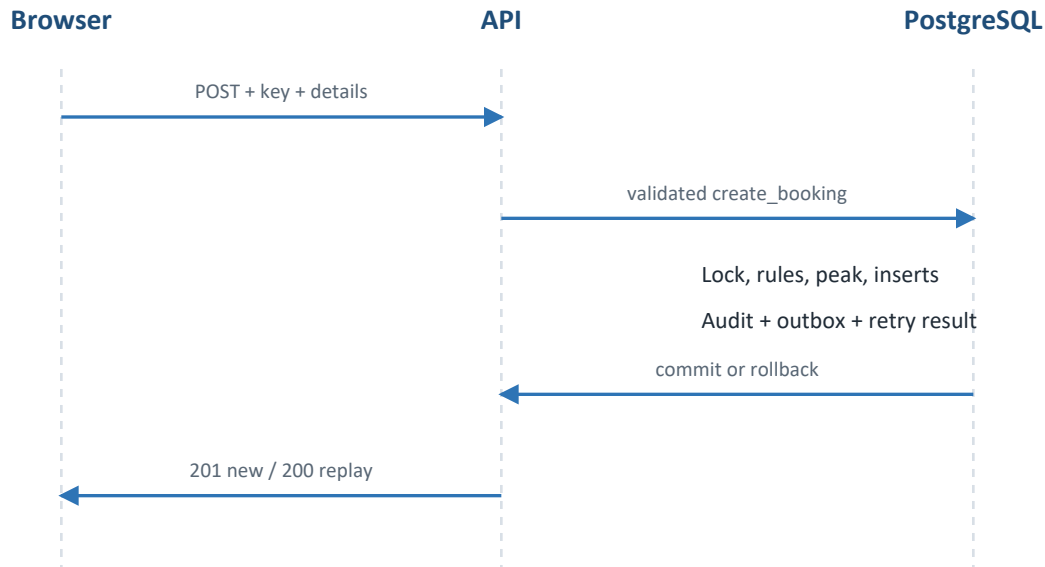
TypeScript types in `shared/types.ts` describe the data client and server exchange. They do not replace runtime validation: a malicious client can send any JSON. Zod checks it on receipt, and PostgreSQL constraints/functions check persistence invariants again.

Use Find in Files to follow one function name, such as `create_booking`. Read its caller, its SQL definition and the integration tests before exploring every page. This turns a large repository into a sequence of small explanations.

Other UI files have clear jobs: Home handles presentation, Menu handles browsing, Account handles Auth, Staff handles protected operational views, Auth maintains client session state, hooks implements reusable loading behavior, and `styles.css` contains the shared responsive styling.

## 9. Follow one booking from click to commit

The form fetches settings and availability. Its chosen start is sent as a UTC ISO timestamp. Local opening rules are interpreted in the cafe timezone. The form sends normalized details and a UUID Idempotency-Key.



In `reservationRoutes`, the POST handler verifies an optional bearer identity, validates the header/body, removes the empty honeypot field and hashes the canonical payload. It creates a deterministic guest credential for an anonymous request and passes its hash to PostgreSQL.

The handler calls `cafe_private.create_booking`. This function locks cafe settings, looks for a saved request, validates time/party/opening rules, and consumes durable booking budgets only for a new request. Inserting a reservation invokes the capacity trigger; its audit and notification triggers run in the same transaction. The request's original response is saved before commit.

If any operation fails, no partial booking/audit/outbox/idempotency result is committed. A new successful request returns HTTP 201. A matching retry returns HTTP 200 and the saved creation result. The management GET retrieves the current record, which may now have a different status.

Important distinction: the availability list is advice. Another customer may book between reading it and submitting. The final database transaction can reject that offered time with `CAPACITY_EXCEEDED`; the UI must let the customer choose again.

## 10. Why simultaneous requests cannot overbook

Imagine capacity four, with a three-person active booking. Twenty people each request the final seat at once through two API instances. The requests reach the same PostgreSQL settings-row lock. The first eligible request inserts its one-person booking and commits. The next request sees occupancy four and is rejected. A lock in one Node process could not coordinate the other instance.

The capacity trigger uses that same lock and checks peak occupancy for every active reservation insert/change affecting capacity. This also blocks an accidental privileged insert that would exceed capacity. Database owners remain trusted administrators and can alter schemas; the threat model cannot make an owner unable to change their own database.

Visits use half-open intervals  $[start, end)$ . A visit ending at 13:30 does not overlap a visit beginning at 13:30. Peak occupancy is the greatest simultaneous total, not the sum of every party whose interval intersects the candidate.

Worked example: an existing two-person visit is 12:00-13:30 and another two-person visit is 13:30-15:00. A candidate 12:30-14:00 intersects both. The existing peak is two, not four, because those existing visits never coexist. `peak_seats` checks the candidate start and each intersecting visit's start, summing parties active at each point.

Pending, confirmed and seated records count toward capacity.

Cancellation/rejection/no-show/completion stop counting. Cancellation locks settings before the booking row; the application follows a consistent order to reduce deadlocks. Availability is uncached, and the database remains authoritative even if a frontend view is stale.

The trade-off is a single cafe-wide lock: simple to prove, limited throughput. Do not claim unlimited scaling. Measure waits and database growth before introducing per-venue locks or slot inventory.

## 11. Retry keys and private guest access

Idempotency means that retrying the same logical request creates one result. The browser keeps the last payload/key in tab session storage, scoped to the signed-in user or guest. Unchanged retries reuse it; changed details get a new key.

PostgreSQL associates the key with the actor and SHA-256 fingerprint. The same key/body/actor returns the original creation response. Different details or a different actor return `IDEMPOTENCY_MISMATCH`. Concurrent duplicates serialize under the lock. Replays do not create extra outbox events or consume the successful-new-booking budget again.

Guest tokens are HMACs derived from the random UUID key and payload hash using a stable server secret. Only the token hash is stored. The copied link uses a URL fragment; the browser sends the token in `X-Booking-Token`, not a query string. Knowing the booking UUID alone is insufficient. Keep the guest secret stable across API instances and releases.

To demonstrate an actual retry against your configured local app, use a second PowerShell terminal. This creates one isolated demo booking, so cancel it afterwards. Choose the returned available slot:

```
$demoDate = (Get-Date).AddDays(7).ToString('yyyy-MM-dd')
$demoUrl = 'http://127.0.0.1:3001/api'
$offer = Invoke-RestMethod "$demoUrl/availability?date=$demoDate&guests=1"
```

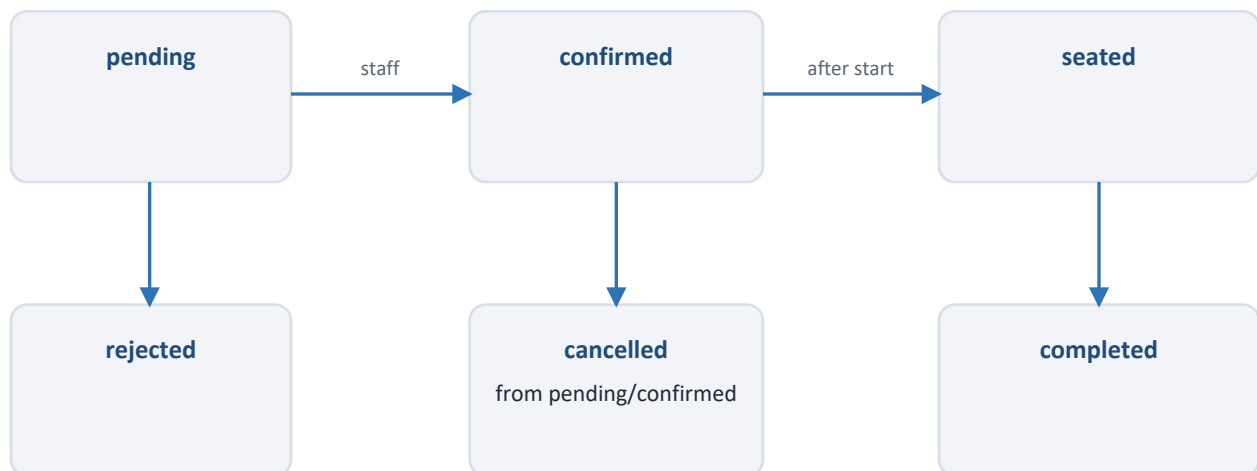
```

$demoSlot = $offer.slots | Where-Object available | Select-Object -First 1
if (-not $demoSlot) { throw 'No available slot; choose another date.' }
$demoKey = [guid]::NewGuid().ToString()
$demoBody = @{
    name='Interview Guest'; email='interview@example.test'
    phone='9876543210'; guests=1; starts_at=$demoSlot.starts_at
    notes='Local demo only'
} | ConvertTo-Json
$demoHeaders = @{ 'Idempotency-Key'=$demoKey }
$first = Invoke-RestMethod "$demoUrl/reservations" -Method Post `
    -Headers $demoHeaders -ContentType application/json -Body $demoBody
$retry = Invoke-RestMethod "$demoUrl/reservations" -Method Post `
    -Headers $demoHeaders -ContentType application/json -Body $demoBody
$first.reservation.id -eq $retry.reservation.id
$retry.replayed

```

Both last outputs should be True. A different body with that same key receives 409; it must not create another booking. The load script also verifies this behavior automatically.

## 12. States, cancellation and stale edits



All new API requests begin pending. Staff can confirm/reject a pending request. Staff can seat a confirmed booking or mark it no\_show after the start time; a seated booking can complete. Owners/guests can cancel pending/confirmed before the configured deadline; staff may cancel those states after the deadline too. Terminal states cannot reopen. Rescheduling is currently cancel-and-create.

Each record has an integer version. A staff member reads version 1, changes the status, and the DB writes version 2. Another staff member who still submits version 1 gets STALE\_VERSION and must refresh. This prevents a stale screen from silently overwriting the newer decision. Even an authorized user must satisfy the state/time/version rules.

The SQL function change\_booking locks and checks the current record, authorizes its actor/token, validates the transition, increments the version, and lets the trigger append audit/outbox entries. Reservation audit includes old/new status, resulting version, timestamp and actor kind/ID. A copied

guest token is recorded as guest authority when used by a signed-in person who does not own that record.

Default rules are development assumptions: Asia/Kolkata, capacity 24, parties up to 10, 90-minute duration, 30-minute starts, daily 09:00-22:00, two-hour lead/cancellation notice and a 60-day horizon. Confirm cafe policy before production. Pending records do not automatically expire. Overnight opening windows and table assignment are not implemented.

Operator settings changes are described in `docs/operations.md`. Lowering capacity below the peak of existing active bookings is rejected. Changing duration does not rewrite historical end times. Review existing records before changing hours/timezone.

## 13. Authentication, authorization and RLS

Authentication answers who the requester is. Authorization answers what that person can do. Supabase Auth issues the session; the API calls `auth.getUser(token)` to verify it and reads the role from `public.profiles`. A browser's session or decoded token claim alone is not server proof.

| Identity           | Allowed core actions                                            |
|--------------------|-----------------------------------------------------------------|
| Public visitor     | Read published menu, inspect availability, create guest request |
| Guest token holder | Read that booking and cancel within its rules                   |
| Customer           | Own records and allowed cancellation; no staff operations       |
| Staff              | Reservation/menu management, real metrics, queue visibility     |
| Administrator      | Staff powers plus reviewed role assignment                      |

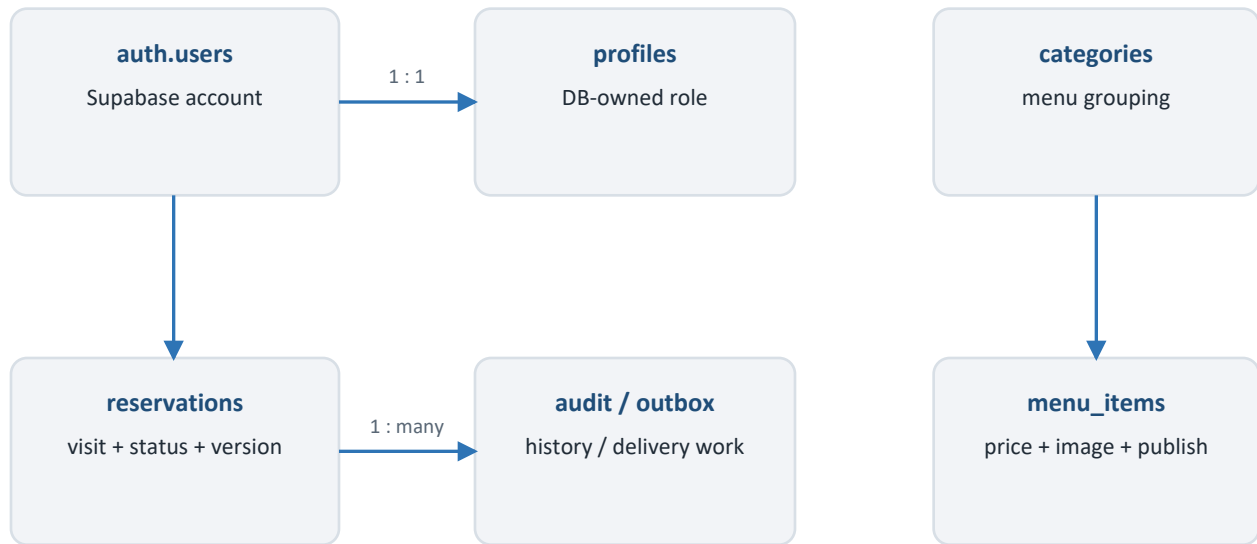
Client UI guards improve navigation, but protected server handlers repeat authorization. The runtime DB role can read operational data and edit menu tables, so its handler checks are essential. It cannot directly write reservations/profiles; only narrowly granted private functions perform those changes.

Publicly exposed tables use Row Level Security and explicit grants. Anonymous/authenticated clients may read intended published menu content. Customers may select their own profile/reservations/audit rows. Direct client mutation grants are absent. Private functions are not exposed/callable by client roles. Booking DML is revoked from `service_role` as well; a Supabase secret key is not the application's booking permission mechanism.

Storage has public reads for menu images. Customer/staff JWTs do not get direct uploads. The staff-authorized server uploads using its secret after checking bytes/pixels and re-encoding.

Sessions live in `tab sessionStorage` and refresh via the SDK. Browser XSS remains a risk for SPA tokens; use HTTPS, controlled script sources and dependency maintenance. Do not claim MFA is implemented merely because Supabase supports it. High-assurance staff deployment should configure and verify it separately.

## 14. Understand the database model



Reservation ownership is optional; guest hashes and retry results are private.

Public tables are the application records exposed with grants/RLS: **profiles** (Auth user role), **categories**, **menu\_items**, **reservations** and **reservation\_audit**. **auth.users** is Supabase-owned; **profiles** refer to its UUID. Reservations optionally refer to a user, or remain guest records. Prices use integer paise: 18000 means INR 180.00.

Private tables implement mechanisms: **settings** contains cafe policy; **guest\_access** stores token hashes; **booking\_requests** stores key/fingerprint/actor/original result; **notification\_outbox** stores delivery work; **request\_limits** stores durable counters; **imported\_sources** detects drift on safe import reruns.

Foreign keys preserve relationships. Category deletion is restricted while items reference it. Auth deletion removes the profile and detaches reservation ownership instead of deleting operational records. Reservation **audit/outbox/request** references prevent casual deletion. This does not automatically anonymize personal details; retention is an operator policy.

Indexes match real queries: GiST on active time ranges for overlaps, start/status ordering for staff pages, user/start for customer pages, reservation/id for audit, and due-state/time for worker claims. A partial published-menu category/name index helps browsing. Simple substring staff searches are paginated; trigram search is a future measured improvement.

The three versioned migrations establish the core schema/rules, the WebP Storage bucket/policies, and validated settings/import checksums. **supabase/seed.sql** inserts labelled development samples safely. Shared TypeScript contracts are maintained in **shared/types.ts**; changing data contracts requires checking both client/server and migration compatibility.

## 15. Menu, images, dashboard and jobs

`server/modules/menu.ts` returns paginated published items with optional category, dietary and search filters. Staff endpoints validate category/item edits and reject stale versions. Public fields use prices in paise and Storage object paths, not arbitrary user-supplied remote image URLs. UI loading, empty, failure and retry states are explicit.

An image upload must pass staff authorization, request size limits and actual decoded image checks. Accepted static JPEG/PNG/WebP input is limited to 3 MiB and 16 megapixels, resized to at most 1200 pixels wide and re-encoded as WebP. SVG, HTML and disguised content are rejected. Metadata is stripped. Unique object paths permit immutable caching. Unreferenced old uploads need reviewed operator cleanup, not automatic deletion.

`server/modules/admin.ts` computes totals from SQL: all reservations, pending count, today's active visits/guests in cafe timezone, published items, queued events and failed events. These are operational counts, not revenue or actual historical customer popularity.

Booking and status triggers write outbox rows in the same transaction as the record/audit change. The worker claims a due event with `FOR UPDATE SKIP LOCKED`, sets a 60-second lease and random token, then calls Resend with a stable event key. Other workers skip that locked row. A crash leaves durable work that can be reclaimed after lease expiry.

Acknowledgement/failure updates require the matching lease token. This fencing prevents a late old worker from overwriting a newer claim. Retries back off exponentially, cap at eight attempts and stop after 23 hours from the first attempt. Resend's documented deduplication window is 24 hours; the project does not promise exactly-once delivery forever. Late ambiguous events become dead for operator review. Delivery order across workers is not guaranteed.

To send booking emails, configure a real `RESEND_API_KEY` and verified `EMAIL_FROM`, then run `npm run worker` in another terminal. Without them the worker exits before claiming, and bookings stay valid with queued events. Auth confirmation/reset mail is configured separately in Supabase. A sent provider receipt is not a guarantee that a customer read the mail.

## 16. Tests and what was actually verified

Run checks in the root. Stop normal development servers before browser tests because their isolated fixture uses ports 3000/3001.

```
npm run typecheck
npm run lint
npm test
npm run build
npm run test:embedded
npx playwright install chromium
npm run test:e2e
npm run load -- --native --requests 40 --concurrency 20
```

Verified in this Windows workspace: type checks, lint and production frontend/server build; six unit tests; 22 integration tests against fresh native PostgreSQL 18.4; six Chromium browser tests across

desktop/mobile; and zero known production dependency advisories at the recorded audit. Browser fixture teardown also left no temporary PostgreSQL processes running.

Integration tests cover last-seat contention across two API instances, overlap/adjacency, duplicate races, mismatch, cancellation, time/state/version checks, unauthorized/cross-user actions, SQL RLS/grants, distributed budgets, image rejection, import dry run/rerun/drift and worker failure/lease recovery. Native test-only Auth/Storage schema shapes and injected identity verification are not a real Supabase HTTP stack.

The recorded load experiment used two localhost APIs, 40 requests at concurrency 20, capacity eight and one guest/request. Eight bookings succeeded, 32 returned expected conflicts, zero had unexpected errors. p50 was 371 ms, p95 617 ms and p99 663 ms. Replay/mismatch/cancel/replacement checks passed. It was a short correctness experiment, not demonstrated production traffic capacity; Auth/network/provider throughput were excluded.

After a real LOCAL Supabase stack is running, set TEST\_SUPABASE\_URL/public key/secret key to the same instance as both DB URLs, then run `npm run test:security`. It makes actual Auth, PostgREST and Storage HTTP access attempts. Missing credentials blocked this check in the recorded workspace. Real provider delivery was also unverified. Remote CI is configured, and its success is evidence only after the workflow actually runs. See `docs/verification.md` for the precise record.

## 17. Troubleshoot by symptom

### Docker or Supabase startup fails

Run `docker info`. If the daemon is unavailable, start Docker Desktop and wait. Re-run `npx supabase start` and inspect its error. Check occupied ports and Docker prerequisites. Do not wipe a database to work around a connection error. Large first-time downloads are expected.

### Frontend works but menu/booking fails

`npm start` launches only Vite. Run the API too, or use `npm run dev`. Check `/api/health/ready` directly at port 3001. Inspect the terminal's safe error/request ID. A readiness 503 usually means the API cannot query its configured database. Restart after environment edits.

### API reports missing configuration or wrong role

Compare `.env` against `.env.example`; remove placeholders. Both `server` and `VITE_public` values are needed. `DATABASE_URL` must use `cafe_api`, matching the password provisioned by `db:role -- --apply`. `ADMIN_DATABASE_URL` is the operator URL for setup scripts. Check that migrations created `cafe_api` in this same database.

### Login says unconfirmed or dashboard says staff access

Follow the local confirmation email. In Studio verify the account UUID and confirmed email. Bootstrap the first administrator with that UUID, then sign out/in or refresh role state. A valid customer login does not imply staff authority. Email input on a booking never grants ownership or role access.

## No available time, 409 or 429

Choose a future local date inside the horizon, a party under the maximum and a time offered by availability. Allow the configured lead time and visit duration before closing. A 409 can mean capacity, stale version, mismatch or invalid transition: use the returned error code. A 429 means a durable rate budget was reached; wait for its window. Retry-After is a short hint, not a guarantee the longer booking budget has expired.

## Upload or email does not work

Uploads need real Supabase Storage, matching server credentials, a staff identity and allowed bytes/size. Email jobs need the separate configured worker. Auth emails use Supabase mail; reservation notifications use Resend. Examine queued/dead status before retrying an ambiguous delivery. Do not claim email arrived just because a booking succeeded.

## Browser tests fail to start or native binaries are blocked

Stop development servers occupying 3000/3001 and install Chromium. Test subprocesses may require permission in restricted Windows environments. The Playwright runner owns the isolated database teardown. Native test directories are ignored local artifacts, not production databases. Tests must never be pointed at live production credentials.

# 18. Deployment, migration and rollback

Production has separate deployment units: static build/, compiled API dist/server/index.js, and compiled worker dist/server/worker.js, plus the Supabase project. Netlify/Vercel frontend configuration does not deploy the API. Vite public values are build-time configuration. A production API URL must reach a deployed backend with matching CORS origin.

Use HTTPS, verified DB TLS, restricted runtime credentials, stable HMAC secrets and a bounded connection budget. Configure actual Auth redirects/mail, Storage and provider domain. Apply migrations through one owner before serving the new API. Use the Dockerfile for a non-root API container and override its command for the worker. Health supervision, backups and monitoring remain operator responsibilities.

The prepared migration is intentionally separate from startup. Export Firestore and Auth through approved operator tooling. Inventory Cloudinary/Firebase Storage files. Keep private exports ignored. User import uses random unknown passwords and a verified reset/linking process; it does not transfer Firebase sessions or assume hash compatibility. No source MongoDB migration is needed because none was found in the active inspected app.

Data/file/user importers validate dry runs and safe reruns. Relational import preserves reviewed statuses/timestamps/relationships, rejects unmapped or changed source records and suppresses notifications. Old guest records cannot be claimed merely by email. Confirm source counts and policy in staging before cutover.

No production export/import, deletion, deployment switch or password transfer was performed by the upgrade. After a real cutover, rollback must preserve new Supabase bookings. Freeze writes, retain both backups and reconcile new records before reopening Firebase writes; otherwise the old system could

overbook unaware of new reservations. Use `docs/migration.md` for the complete procedure, not an improvised reset command.

## 19. Practise a five-minute interview demo

1. Open the configured app and show mobile navigation, menu filters and the sample-data label. Explain React + API + Supabase in one sentence.
2. Book as a guest, save the private link and show pending status. In Studio point at the reservation, audit and outbox row. Explain why email is independent of the successful booking commit.
3. Run `npm run test:embedded` and point at the twenty-request last-seat test: exactly one success across two instances. Explain the DB lock and why availability is only advisory.
4. Run the retry example in chapter 11 or show the duplicate test. Same key/body yields the same ID; changed body fails. Cancel the demo booking to release its allocation.
5. Sign in as administrator/staff, search and confirm a request, show history, edit/publish a menu item and inspect real metrics/notification states. Demonstrate image upload only with working real Storage.

If the provider is configured, show the worker and receipt. Otherwise show queued work and the recovery test while stating delivery is unverified. Give the measured workload and latency with its exclusions; do not turn a 40-request experiment into a traffic-capacity claim.

Your short explanation can be: The browser requests a booking, but PostgreSQL decides seat allocation. A transaction locks cafe capacity, validates overlap and saves the reservation, audit, retry result and notification event together. Server-verified roles guard staff operations. Email is delivered later by a retryable worker.

Practise answering why a seat-based model differs from table assignment; why pending records need an operational policy; why a role in user metadata is unsafe; why RLS also needs grants; why versions matter even with authorization; and why external delivery is not exactly-once forever. The code/tests demonstrate each answer more reliably than memorized claims.

## 20. Check your understanding and keep learning

### Questions you should answer without reading

1. Where does a booking become authoritative? Answer: the PostgreSQL transaction, not the availability response or form.
2. What stops two servers taking the final seat? Answer: the shared settings-row lock and capacity trigger/peak check.
3. What happens after a lost HTTP response? Answer: retry the same key/body/actor and receive the stored creation result.
4. Why can a confirmed customer still fail a staff request? Answer: authentication succeeds but DB-owned role authorization does not.

5. What happens when staff edit the same version? Answer: one changes/increments it; the stale update is rejected.
6. What if email fails after booking? Answer: the booking remains; durable outbox work retries or becomes visible as dead.
7. Can a guest recover by entering the original email? Answer: no automatic ownership claim exists; the private token is required.
8. What was not proven locally? Answer: real Supabase Auth/Storage/Data API behavior, provider delivery and production capacity.

## A practical study sequence

Session 1: run the full app, make/cancel a guest booking and identify each process. Session 2: trace Booking.tsx to reservationRoutes/create\_booking; explain UTC versus local rules. Session 3: study locking, peak occupancy and duplicate tests. Session 4: trace token verification, roles, grants/RLS and versioned staff changes. Session 5: study outbox leases, migration safety and measured limits; give the demo aloud. Move on when you can explain and reproduce each outcome, not merely after reading it.

Make one small change on another branch, such as a sample menu description or a validated opening setting. Predict its effect, run appropriate checks and explain why it does not require rewriting the booking invariant. Avoid changing a checksum-tracked migration after it has been applied; use a new migration.

## Glossary

Transaction: database operations that commit or roll back together. Row lock: a database-held turn to modify/check shared state. Idempotency: the same request yields one recorded result. Fingerprint: a hash identifying normalized request details. RLS: per-row access policies for a database role. Grant: permission to use a table/function/schema. Outbox: delivery work stored with the business commit. Lease: temporary claim that expires after a worker crash. Fencing token: a value that rejects updates from an outdated worker. Optimistic version: a counter used to detect stale edits. Paise: integer hundredths of a rupee. Modular monolith: one service codebase organized into focused modules.

## Reference pages and commands

Project references: README.md, docs/api.md, docs/architecture.md, docs/system-design.md, docs/operations.md, docs/migration.md, docs/interview.md and docs/verification.md. They provide route examples, full design trade-offs and operator procedures beyond this guided reading sequence.

If an upgrade commit is prepared locally but not yet published, publish its branch after reviewing staged changes and confirming no exports/secrets are included:

```
git status --short
git diff --cached --stat
git push -u origin upgrade/supabase-reliable-reservations
```

If there is no upgrade commit, review and stage the intended upgrade files, then commit before pushing. Exclude .env, private exports, local databases/backups, the independent nested checkout and unrelated

dirty hosting cache. Do not blindly stage every uncommitted file. After pushing, select the upgrade branch on GitHub. Merge into main only after review and required checks, particularly real Supabase checks. The old fixed commit URL will still display the old code.

Official references for continued study:

- [Supabase local CLI setup](#)
- [Supabase server user verification](#)
- [Supabase Row Level Security](#)
- [Supabase Storage access control](#)
- [PostgreSQL explicit locking](#)
- [Resend idempotency keys](#)
- [Firebase Auth migration considerations](#)

This guide is generated from its Markdown source by `docs/guide/build_guide.py`. Rebuild/render instructions are in `docs/guide/README.md`. The PDF includes searchable text, navigation bookmarks, diagrams and page numbers.